

City of Karis

PHASE 1 · TECHNICAL HANDOVER DOCUMENT

Version: 1.0

Date: 2026-04-27

Status: Production-Ready

Prepared by: White Dragon Development

Audience: Senior Developer (external handover)

K Credits · Resident Portal · Admin Back-Office
Next.js 16 · Prisma v7 · Clerk · Supabase · Vercel

City of Karis — Phase 1 Technical Handover

Document version: 1.0

Last updated: 2026-04-27

Built by: Claude Sonnet 4.6 (claude-sonnet-4-6) across 8 sessions on 2026-04-26

Audience: External senior developer taking over maintenance and operation

Table of Contents

- [1. Project at a glance](#)
- [2. Quick start for a new developer](#)
- [3. Technology stack](#)
- [4. Architecture overview](#)
- [5. Repository structure](#)
- [6. Domain model](#)
- [7. The K Credit ledger — critical reference](#)
- [8. Authentication and authorization](#)
- [9. Feature catalog](#)
- [10. Brand system](#)
- [11. Operations runbook](#)
- [12. Deployment and environments](#)
- [13. Common tasks — how to](#)
- [14. Known issues and technical debt](#)
- [15. Path to Phase 2](#)
- [16. Contacts and access](#)
- [17. Appendix A: Full Prisma schema](#)
- [18. Appendix B: Environment variables reference](#)
- [19. Appendix C: API surface](#)
- [20. Appendix D: Useful queries](#)

1. Project at a glance

The City of Karis Community App is the resident-facing portal and admin back-office for the City of Karis settlement — a planned residential community in Guyana. Residents hold **K Credits**, an internal store-of-value token pegged 1:1 to USD and backed by a treasury reserve. They can spend K Credits with vendors inside the settlement, request cash settlements, track property ownership and installment payments, and participate in community governance (votes, updates, issues). Admins manage accounts, treasury deposits, settlement approvals, property records, and community content.

Phase 1 is **production-ready**: 26 compiled routes, 36 passing unit tests, a fully migrated PostgreSQL schema, and a working seed dataset. The build was completed across 8 sessions on 2026-04-26 using Next.js 16.2.4 on Vercel, Supabase for the database, and Clerk for authentication.

Resource	URL
Production app	https://YOUR_VERCEL_URL (fill in after deployment)
GitHub repo	https://github.com/YOUR_USERNAME/cok-app (fill in after push)

Supabase dashboard	https://supabase.com/dashboard
Clerk dashboard	https://dashboard.clerk.com
UploadThing dashboard	https://uploadthing.com/dashboard
Resend dashboard	https://resend.com/overview
Vercel dashboard	https://vercel.com/dashboard

2. Quick start for a new developer

Estimated time: 20–30 minutes from zero to running dev server.

Prerequisites

- Node.js 20+ and pnpm 10+
 - `node --version` should print `v20.x` or higher
 - `pnpm --version` should print `10.x` or higher
- Git
- Access to credentials for: Supabase, Clerk, UploadThing, Resend (see [Appendix B](#))

Step 1 — Clone and install

```
git clone https://github.com/YOUR_USERNAME/cok-app.git
cd cok-app/website
pnpm install
```

Expected output ends with something like:

```
Progress: resolved 847, reused 847, downloaded 0, added 847, done
```

Important: After install, run `pnpm exec prisma generate` once to build the Prisma client. Installing packages can invalidate the generated client in Prisma v7.

```
pnpm exec prisma generate
```

Expected: `Prisma Client` generated successfully.

Step 2 — Configure environment variables

```
cp .env.example .env
```

Open `.env` and fill in every value. See [Appendix B](#) for where to find each one.

Critical notes:

- `DATABASE_URL` and `DIRECT_URL` must both point to the **session pooler** (port **5432**), not the transaction pooler (port 6543). Transaction mode blocks prepared statements that Prisma uses.
- Both `DATABASE_URL` and `DIRECT_URL` can be identical in this setup (they were set to the same URL during the original build).
- The app will throw `Environment validation failed: ...` at server start if any required variable is missing or empty.

For local development with Clerk, you also need `.env.local` alongside `.env` :

```
cp .env.example .env.local
# fill in .env.local with the same values
```

Prisma CLI reads `.env` ; Next.js reads `.env.local` . Keep both in sync.

Step 3 — Run migrations

```
pnpm exec prisma migrate deploy
```

Expected output:

```
Applying migration `20260426080152_init`
The following migration(s) have been applied:

migrations/
├─ 20260426080152_init/
└─ migration.sql

All migrations have been applied.
```

If you see `P3009 migrate found failed migrations` or drift errors, run:

```
pnpm exec prisma migrate reset --force
pnpm exec prisma migrate deploy
```

This wipes and recreates the schema. Safe for dev only.

Step 4 — Seed the database

For development (system wallets + 6 demo users + fee schedule + demo data):

```
pnpm seed
```

Expected output includes:

```
Seeding City of Karis database...
System wallet: community_fund
System wallet: operations_fund
System wallet: developer_share
System wallet: treasury_reserve
System wallet: settlement_burn
User: karis@cityofkaris.com (MASTER_ADMIN)
```

```
User: devon@example.com (RESIDENT)
...
```

For production (system wallets + fee schedule only — no demo users):

```
pnpm seed:production
```

Step 5 — Start the dev server

```
pnpm dev
```

Expected output:

```
▲ Next.js 16.2.4
- Local:      http://localhost:3000
- Network:    ...
Ready in ~470ms
```

Open `http://localhost:3000` . You will see the landing page with the COK hero.

Note: If you see `Publishable key not valid.` in the browser, your Clerk keys are wrong or missing in `.env.Local` . The app requires real Clerk credentials to render any page.

Step 6 — Sign in

Use the Clerk sign-in flow at `http://localhost:3000/sign-in` . After first sign-in, the Clerk webhook creates your user row in the database. To test as the seeded Master Admin, you need to sign in with the email that matches the `karis@cityofkaris.com` user and then ensure the Clerk webhook has set the `clerkId` field on that user row (or set it manually via Supabase table editor).

Quick verification commands

```
pnpm typecheck # Should report: 0 errors
pnpm lint      # Should report: 0 errors, 0 warnings
pnpm test      # Should report: 36 passed | 36 total
pnpm build     # Should report: 26 routes compiled
pnpm exec prisma validate # Should report: Schema valid
```

3. Technology stack

Framework

Package	Version	Purpose	Docs
next	16.2.4	App Router, SSR/RSC, routing	https://nextjs.org/docs
react	19.2.4	UI library	https://react.dev

react-dom	19.2.4	DOM renderer	https://react.dev
typescript	^5	Type safety	https://www.typescriptlang.org

Database

Package	Version	Purpose	Docs
prisma	7.8.0	ORM, schema management, migrations	https://www.prisma.io/docs
@prisma/client	7.8.0	Type-safe database client	https://www.prisma.io/docs/concepts/components/prisma-client
@prisma/adapters-pg	7.8.0	PostgreSQL driver adapter for Prisma v7	https://www.prisma.io/docs/orm/overview/databases/postgresql
pg	8.20.0	Raw PostgreSQL node.js driver	https://node-postgres.com

Prisma v7 breaking change: Connection URL is no longer in `schema.prisma`. It lives in `prisma.config.ts` via `defineConfig`. The `@prisma/adapters-pg` adapter is instantiated in `lib/db.ts`.

Auth

Package	Version	Purpose	Docs
@clerk/nextjs	7.2.7	Authentication, user management, webhooks	https://clerk.com/docs/nextjs
svix	1.92.2	Webhook signature verification (used in Clerk webhook handler)	https://docs.svix.com

Storage & email

Package	Version	Purpose	Docs
uploadthing	7.7.4	File upload service (photos, PDFs, proof documents)	https://docs.uploadthing.com
@uploadthing/react	7.3.3	React hooks and components for UploadThing	https://docs.uploadthing.com/api-reference/react
resend	6.12.2	Transactional email API	https://resend.com/docs

UI

Package	Version	Purpose	Docs
tailwindcss	^4	Utility-first CSS (v4, CSS-only config)	https://tailwindcss.com/docs

shadcn	4.5.0	Component generator/registry	https://ui.shadcn.com/docs
@base-ui/react	1.4.1	Headless UI primitives (Select, Menu, etc.)	https://base-ui.com/docs
class-variance-authority	0.7.1	Variant-based className builder	https://cva.style
clsx	2.1.1	Conditional className utility	https://github.com/lukeed/clsx
tailwind-merge	3.5.0	Merge Tailwind classes without conflicts	https://github.com/dcastil/tailwind-merge
tw-animate-css	1.4.0	Animation utilities for Tailwind v4	—
lucide-react	1.11.0	Icon library (only permitted icon source)	https://lucide.dev
next-themes	0.4.6	Theme switching	https://github.com/pacocoursey/next-themes
sonner	2.0.7	Toast notifications (replaces deprecated shadcn toast)	https://sonner.emilkowal.ski

Forms & validation

Package	Version	Purpose	Docs
react-hook-form	7.74.0	Form state management	https://react-hook-form.com
@hookform/resolvers	5.2.2	Zod validation adapter for RHF	https://github.com/react-hook-form/resolvers
zod	4.3.6	Schema validation (forms + env)	https://zod.dev

Financial math

Package	Version	Purpose	Docs
decimal.js	10.6.0	Arbitrary-precision decimal math	https://mikemcl.github.io/decimal.js

Note: Prisma.Decimal re-exports decimal.js 10.x. Code uses Prisma.Decimal exclusively to avoid type mismatches between two Decimal class instances.

PDF & media

Package	Version	Purpose	Docs
@react-pdf/renderer	4.5.1	PDF generation (transaction receipts, installment receipts)	https://react-pdf.org
qrcode.react	4.2.0	QR code rendering (member ID card)	https://github.com/zpao/qrcode.react

Date utilities

Package	Version	Purpose	Docs
---------	---------	---------	------

date-fns	4.1.0	Date formatting, relative dates	https://date-fns.org
----------	-------	---------------------------------	---

Dev tools

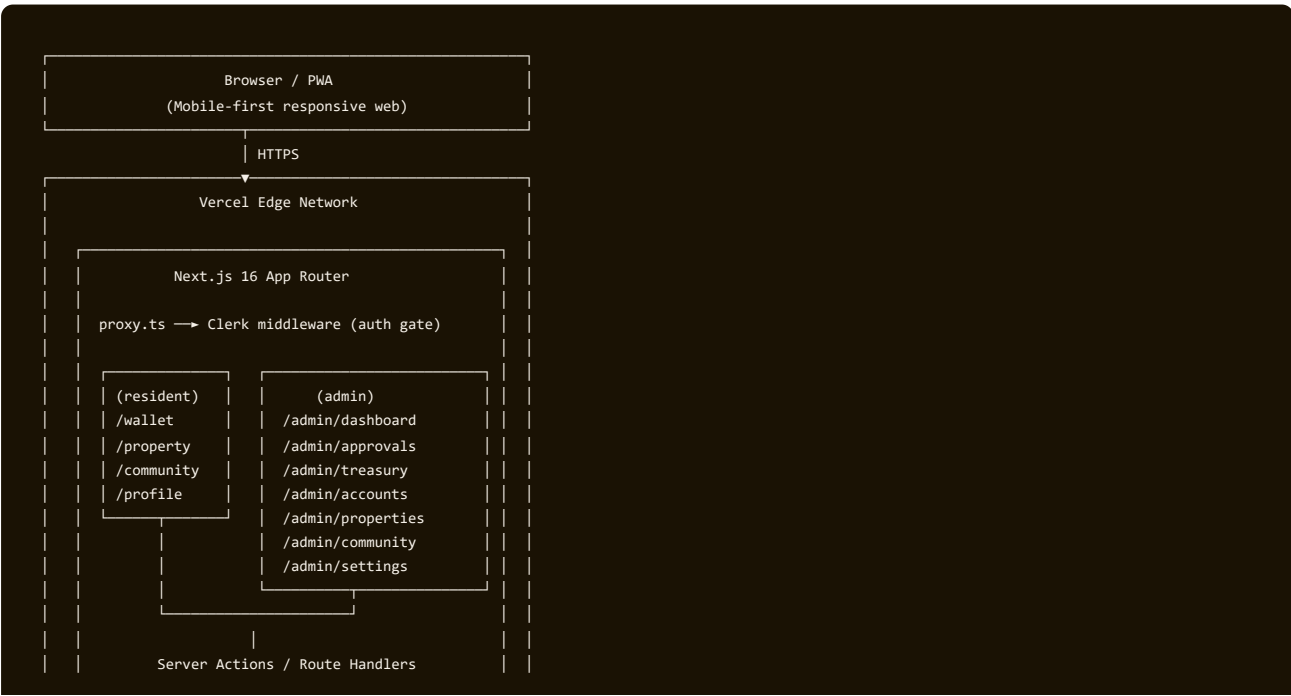
Package	Version	Purpose	Docs
vitest	^4.1.5	Unit test runner	https://vitest.dev
@vitest/coverage-v8	^4.1.5	Code coverage	https://vitest.dev/guide/coverage
tsx	^4.21.0	TypeScript execution for seed scripts	https://github.com/privatenumber/tsx
eslint	^9	Linters	https://eslint.org
eslint-config-next	16.2.4	Next.js ESLint rules	—
playwright-core	^1.59.1	Browser automation (installed, not yet wired to tests)	https://playwright.dev

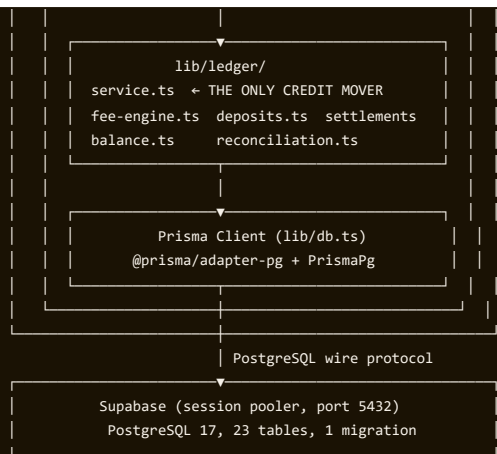
Hosting

Service	Purpose
Vercel	Production hosting, CI/CD, edge functions
Supabase	PostgreSQL database (managed, with automated backups)

4. Architecture overview

System layers





External services:

Clerk ← proxy.ts auth gate + webhook sync (POST /api/webhooks/clerk)
 UploadThing ← file uploads (photos, PDFs) via /api/uploadthing
 Resend ← transactional email (lib/notifications/service.ts, not yet fully wired)

Data flow: user sign-up

1. User fills Clerk <SignUp /> form → Clerk creates a ClerkUser
2. Clerk fires webhook POST /api/webhooks/clerk (user.created event)
3. Webhook handler verifies svix signature with CLERK_WEBHOOK_SECRET
4. Handler calls db.user.create({ memberId, email, fullName, role: VISITOR })
5. Handler calls db.wallet.create({ userId }) – empty wallet provisioned
6. Next request: proxy.ts calls auth.protect() → Clerk confirms session
7. lib/auth.ts getCurrentUser() queries db.user WHERE clerkId = clerkUserId
8. App serves the (resident) layout with the user's wallet data

Data flow: recording a deposit (admin)

1. Admin fills deposit form in /admin/treasury, submits
2. recordDepositAction() Server Action called
3. requireRole(['MASTER_ADMIN', 'ADMIN']) confirms admin session
4. lib/ledger/deposits.ts recordDeposit({ userId, fiatAmount, paymentMethod, ... })
5. db.\$transaction():
 - a. Creates Transaction row (type: DEPOSIT)
 - b. Creates LedgerEntry +amount on user wallet
 - c. Creates LedgerEntry -amount on treasury_reserve wallet
 - d. Creates Deposit audit row
6. In development: reconcileTreasury() runs and throws if not balanced
7. revalidatePath('/admin/treasury') – Next.js cache invalidated
8. UI re-renders showing updated balances

Data flow: settlement request (resident → admin → execute)

1. Resident submits settlement form → requestSettlementAction()
2. lib/ledger/settlements.ts requestSettlement() creates SettlementRequest row (PENDING_APPROVAL)
3. Admin sees request in /admin/approvals → approveSettlementAction()
4. approveSettlement() updates status to APPROVED, sets approvedBy + approvedAt
5. Notification created for resident (SETTLEMENT_APPROVED)
6. Admin goes to /admin/treasury → executeSettlementAction(settlementId, proofUrl)
7. executeSettlement() calls transferCredits() from user wallet → settlement_burn wallet
8. Fee applied per fee schedule (RESIDENT_SETTLEMENT type)

9. SettlementRequest updated: status=SETTLED, settledAt, proofUrl, transactionId
10. Notification created for resident (SETTLEMENT_SETTLED)

Data flow: community update (admin publishes → resident sees)

1. Admin fills new update form in /admin/community → publishUpdateAction()
2. db.communityUpdate.create({ category, headline, message, photoUrl, publishedBy })
3. notifyAllOfRole('RESIDENT') creates Notification rows for all RESIDENT accounts
4. Resident opens /community – UpdateAcknowledgement created on "Mark read" tap
5. Unread count appears as gold dot on Community tab in bottom tab bar

5. Repository structure

```
COK-City-of-Karis/
├─ brand_assets/
├─ reference/
└─ website/
    ├─ .env
    │   └─ .env.example
    │   └─ .env.local
    │   └─ .gitignore
    │   └─ proxy.ts
    │   └─ next.config.ts
    │   └─ prisma.config.ts
    │   └─ package.json
    │   └─ pnpm-lock.yaml
    │   └─ vitest.config.ts
    │   └─ tsconfig.json
    │   └─ eslint.config.mjs
    │   └─ components.json
    │   └─ README.md
    │   └─ OPERATIONS.md
    │   └─ AGENTS.md
    │   └─ BUILD_LOG.md
    │   └─ HANDOVER.md
    │
    └─ app/
        └─ layout.tsx
        └─ page.tsx
        └─ globals.css
        └─ manifest.ts
        │
        └─ (auth)/
            └─ layout.tsx
            └─ sign-in/[...sign-in]/page.tsx
            └─ sign-up/[...sign-up]/page.tsx
        │
        └─ (admin)/
            └─ layout.tsx
            └─ _actions/
                └─ settlements.ts
                └─ deposits.ts
                └─ treasury.ts
                └─ accounts.ts
                └─ properties.ts
                └─ community.ts
            └─ admin/
                └─ dashboard/
                └─ approvals/
                └─ treasury/
                └─ accounts/
                └─ properties/
                └─ community/
                └─ settings/
```

← outer project wrapper (not the git root)

← logo PNG, brand PDFs, reference documents

← playbook, wireframes, build instructions

← THE NEXT.JS APP – git repo lives here

← DB credentials (read by Prisma CLI)

← template for all required env vars

← all secrets (read by Next.js; gitignored)

← Clerk auth middleware (Next.js 16: was middleware.ts)

← Next.js config: Turbopack root, image domains

← Prisma v7 connection config (DATABASE_URL)

← deps, scripts (seed, test, build, etc.)

← test runner: Node env, @/ alias, server-only mock

← shadcn/ui registry config

← quick start + architecture summary

← daily/weekly/monthly operational checklists

← Next.js 16 AGENTS note for AI coding agents

← chronological build log for all 8 sessions

← this document

← Next.js App Router

← root layout: ClerkProvider, 3 fonts, PWA manifest

← landing page + role-based redirect

← brand tokens (OKLCH palette), Tailwind v4 config

← PWA manifest (Next.js 16 native)

← sign-in / sign-up (Clerk forms, brand styling)

← admin route group (URL prefix: /admin/*)

← requireRole('MASTER_ADMIN'), AdminSidebar

← Server Actions for admin mutations

← approve/decline/execute settlement

← record deposit

← treasury adjustment

← create/suspend/restore/upgrade account

← create property, add installment, assign owner/tenant

← publish update, create vote, reply to issue

← actual route directories (see route conflict note)

← /admin/dashboard

← /admin/approvals

← /admin/treasury + /admin/treasury/debug

← /admin/accounts

← /admin/properties + /admin/properties/[propertyId]

← /admin/community

← /admin/settings

- └─ (resident)/
 - └─ layout.tsx ← resident/visitor route group
 - └─ _actions/ ← requireRole(['RESIDENT','VISITOR',...]), ResidentTabBar
 - └─ wallet.ts ← Server Actions for resident mutations
 - └─ community.ts ← request/cancel settlement, load more transactions
 - └─ profile.ts ← acknowledge update, cast vote, raise issue, mark notifications
 - └─ wallet/ ← update introduction, update profile photo
 - └─ receipt/[txId]/ ← /wallet, /wallet/transactions, /wallet/settlements
 - └─ receipt/[transactionId] ← /wallet/receipt/[transactionId] – PDF download
 - └─ property/ ← /property
 - └─ receipt/[id]/ ← /property/receipt/[installmentId] – PDF download
 - └─ community/ ← /community + /community/issues
 - └─ profile/ ← /profile
- └─ about/founders/ ← /about/founders (public page)
- └─ access/ ← /access (public)
- └─ login/ ← /login (public)
- └─ dev-login/ ← /dev-login (public, development only)
- └─ api/
 - └─ webhooks/clerk/ ← POST webhook: user.created / user.updated
 - └─ uploadthing/ ← GET + POST handlers for file upload
- └─ components/
 - └─ admin/
 - └─ stat-card.tsx ← admin-specific reusable components
 - └─ page-header.tsx ← dashboard stat card with optional link
 - └─ empty-state.tsx ← consistent h1 + subtitle + action slot
 - └─ k-amount.tsx ← branded empty state with icon
 - └─ k-amount.tsx ← "K 1,250.00" formatted currency display
 - └─ resident/
 - └─ wallet-skeleton.tsx ← loading skeleton for wallet page
 - └─ shared/
 - └─ admin-sidebar.tsx ← used across admin + resident
 - └─ admin-sidebar.tsx ← left nav bar for admin area
 - └─ resident-tab-bar.tsx ← bottom tab bar for resident area
 - └─ brand-logo.tsx ← COK logo from /public/logo.png
 - └─ wordmark.tsx ← "City of Karis" wordmark in Cormorant Garamond
 - └─ ios-install-prompt.tsx ← iOS PWA "Add to Home Screen" prompt
 - └─ ui/
 - └─ shadcn/ui primitives (15 components)
 - └─ [alert-dialog, avatar, badge, button, card, checkbox, command, dialog, dropdown-menu, input, input-group, label, popover, scroll-area, select, separator, sheet, skeleton, sonner, table, tabs, textarea]
- └─ lib/
 - └─ auth.ts ← getCurrentUser(), requireRole(), getClerkUser()
 - └─ db.ts ← Prisma singleton (PrismaPg adapter, HMR-safe)
 - └─ env.ts ← Zod env validation – throws at boot if missing vars
 - └─ member-id.ts ← generateUniqueMemberId() – K-XXXXXX format
 - └─ uploadthing.ts ← UploadButton / UploadDropzone generator
 - └─ utils.ts ← cn() tailwind merge helper
 - └─ ledger/
 - └─ types.ts ← THE FINANCIAL ENGINE (see Section 7)
 - └─ types.ts ← TS interfaces for all ledger types
 - └─ fee-engine.ts ← getActiveFeeSchedule(), calculateFee()
 - └─ service.ts ← transferCredits() – ONLY credit mover
 - └─ deposits.ts ← recordDeposit() – mint from treasury
 - └─ settlements.ts ← full settlement lifecycle
 - └─ balance.ts ← balance queries, formatKCredit()
 - └─ reconciliation.ts ← reconcileTreasury() – invariant check
 - └─ _tests/ ← 36 unit tests (Vitest)
 - └─ notifications/
 - └─ service.ts ← notify(), notifyMany(), notifyAllOfRole()
 - └─ pdf/
 - └─ installment-receipt.tsx ← @react-pdf A5 receipt template
 - └─ queries/
 - └─ dashboard.ts ← server-side read queries (not mutations)
 - └─ dashboard.ts ← treasury + stat data for admin dashboard
 - └─ accounts.ts ← user list, user detail
 - └─ properties.ts ← property list, detail, resident property
 - └─ community.ts ← updates, votes, issues, notifications
 - └─ wallet.ts ← recent transactions, pagination, settlements
 - └─ seed/
 - └─ seed.ts ← system wallets + 6 users + fee schedule
 - └─ demo-transactions.ts ← 5 demo operations
 - └─ history-transactions.ts ← 30 backdated transactions (Devon + Aaliyah)
 - └─ seed-production.ts ← infra-only seed (safe for production)
- └─ prisma/
 - └─ schema.prisma ← 23 models, 8 enums (see Appendix A)
 - └─ migrations/

```
├── 20260426080152_init/
│   └── migration.sql  ← single migration, full schema
├── public/
│   └── logo.png        ← COK metallic gold logo (no background)
├── _mocks_/
│   └── server-only.ts  ← empty export to silence server-only in Vitest
└── collateral/         ← demo scripts, brief (not in git build)
    ├── loom-admin-walkthrough.md
    ├── loom-resident-walkthrough.md
    └── phase1-demo-brief.md
```

6. Domain model

Enums

Enum	Values	Used by
Role	MASTER_ADMIN, ADMIN, VENDOR, RESIDENT, VISITOR	User
AccountStatus	ACTIVE, SUSPENDED, PENDING_KYC	User
TransactionType	DEPOSIT, PURCHASE, TRANSFER, BARTER, PAYROLL, RESIDENT_SETTLEMENT, VENDOR_SETTLEMENT, VISITOR_SETTLEMENT, TREASURY_ADJUSTMENT, FEE_SPLIT, REVERSAL	Transaction, LedgerEntry
SettlementStatus	PENDING_APPROVAL, APPROVED, DECLINED, SETTLED, CANCELLED	SettlementRequest
PropertyType	OWNERSHIP, RENTAL, ADMIN	Property
PropertyCategory	COMMERCIAL, RESIDENTIAL, MIXED	Property
IssueLevel	YELLOW, ORANGE, RED	Issue
IssueStatus	OPEN, IN_PROGRESS, RESOLVED, CLOSED	Issue

Models

Core identity

User — the central account record. One user per Clerk account. Created by the Clerk webhook on first sign-in (or pre-created by admin via invitation). Key fields: `clerkId` (nullable — set when user first signs in), `memberId` (K-XXXXXX format, unique), `role`, `status`, `kyc` (JSON blob for identity documents), `introduction`, `profilePhotoUrl`.

Wallet — each user has exactly one wallet. System wallets (`isSystem: true`) have a `systemKey` instead of a `userId`. The five system wallets are: `community_fund`, `operations_fund`, `developer_share`, `treasury_reserve`, `settlement_burn`.

Ledger

FeeSchedule — a point-in-time fee configuration. `rules` is a JSON object keyed by `TransactionType`, each with `totalPct`, `communityFundPct`, `operationsFundPct`, `developerSharePct`. The active schedule is the one with the highest `effectiveAt` $\leq$ `now`.

Transaction — represents one business event (deposit, purchase, settlement execution, etc.). Has a `type`, `description`, optional `reference`, and links to the `FeeSchedule` active at creation time.

LedgerEntry — a single debit or credit line on a wallet within a transaction. `amount` is positive for credits, negative for debits.
Append-only: never updated or deleted.

Financial operations

Deposit — audit record for fiat → K Credit conversion. Links to the `Transaction` that minted the credits. Stores `fiatAmount`, `currency`, `paymentMethod`, `proofUrl`, `recordedBy`.

SettlementRequest — K Credits → fiat cash-out request. Lifecycle: `PENDING_APPROVAL` → `APPROVED` → `SETTLED` (or `DECLINED` / `CANCELLED`). The ledger debit happens only at execute time (`SETTLED`).

TreasuryAdjustment — audit record for manual treasury corrections. Stored separately from the ledger transaction for additional auditability.

Property

Property — a real estate asset. `code` is unique (e.g., `RESIDENCE-A12`). Can have multiple ownerships (for fractional ownership) and multiple installments. `specifications` and `documents` are JSON blobs.

PropertyOwnership — links a user to a property as an owner, with `ownershipPct` (default 100).

PropertyInstallment — scheduled payment milestone. Has `number`, `dueDate`, `amount`, optional `progressNote`.

PropertyPayment — records a payment against an installment for an ownership. Stores `paidAt` and `proofUrl`.

PropertyTenancy — links a user to a property as a tenant. Has `cycle` (e.g., "monthly"), `cyclePayment`, contract details.

TenancyCyclePayment — records each rental cycle payment.

Community

CommunityUpdate — admin-published announcement. `category`, `headline`, `message`, optional `photoUrl`.

UpdateAcknowledgement — resident's acknowledgement of an update. Composite unique (`updateId`, `userId`).

Vote — a community ballot. Has multiple `VoteOption` children. `isOpen` controls whether voting is active.

VoteOption — one selectable option within a vote.

VoteSubmission — a resident's vote. Composite unique (`voteId`, `userId`) enforces one vote per person per ballot.

Issue — a resident-raised concern with `seriousness` and `urgency` levels, `category`, and a `status` lifecycle.

IssueReply — admin or resident reply to an issue thread.

Notification — in-app notification for a user. `type` is a string (e.g., `SETTLEMENT_APPROVED`, `COMMUNITY_UPDATE`). `readAt` is null until read.

Role permissions matrix

Action	MASTER_ADMIN	ADMIN	VENDOR	RESIDENT	VISITOR
View own wallet	✓	✓	✓	✓	✓
Request settlement	—	—	✓	✓	✓
View property tab	—	—	—	✓	—
Raise issue	—	—	—	✓	✓
Cast vote	—	—	—	✓	✓

Acknowledge update	—	—	—	✓	✓
View admin area	✓	✓	—	—	—
Approve/decline settlement	✓	✓	—	—	—
Execute settlement	✓	—	—	—	—
Record deposit	✓	✓	—	—	—
Create account	✓	✓	—	—	—
Suspend/restore account	✓	✓	—	—	—
Change user role	✓	✓	—	—	—
Create/manage properties	✓	✓	—	—	—
Publish community update	✓	✓	—	—	—
Create/close vote	✓	✓	—	—	—
Reply to / assign issue	✓	✓	—	—	—
Treasury adjustment	✓	—	—	—	—
View treasury debug	✓	—	—	—	—
Upload proof-of-payment docs	✓	✓	—	—	—

Note: `MASTER_ADMIN` is the only role that can execute settlements and make treasury adjustments. There is exactly one `MASTER_ADMIN` in the system. It must be set manually in the database — the UI cannot assign this role.

7. The K Credit ledger — critical reference

This section is the most important in the document. Read it before touching any financial code.

The append-only double-entry principle

Every financial movement in the system is recorded as a set of `LedgerEntry` rows linked to a single `Transaction` row. The invariant is strict:

Every debit has a matching credit. The sum of all `LedgerEntry` amounts across all wallets is always zero — with one exception: `TREASURY_ADJUSTMENT` entries are one-sided (they create credits from nothing, modeling the injection of fiat-backed K Credits into the system).

`LedgerEntry` rows are **never updated and never deleted**. Mistakes are corrected by creating a compensating entry, not by modifying the original.

Single-service-controls-all-credit-movement principle

`transferCredits()` in `lib/ledger/service.ts` is the only function permitted to write `LedgerEntry` and `Transaction` rows in normal operation. No server action, no API route, no background job should write these rows directly. The sole exceptions are:

- `recordDeposit()` in `lib/ledger/deposits.ts` — which uses its own `db.$transaction()` but follows the same invariants
- The seed scripts (`lib/seed/`) — which write historical data directly only when bootstrapping demo content

Fee schedule architecture and the active fee table

Fee schedules are stored as rows in the `FeeSchedule` table. Each row has an `effectiveAt` timestamp and a `rules` JSON blob.

The active schedule is the row with the highest `effectiveAt` that is less than or equal to `now`. This means you can schedule a future fee change by inserting a new row with a future `effectiveAt`.

The `rules` JSON structure is:

```
{
  "PURCHASE": {
    "totalPct": 2.5,
    "communityFundPct": 1.5,
    "operationsFundPct": 0.5,
    "developerSharePct": 0.5
  },
  "BARTER": {
    "totalPct": 1.0,
    "communityFundPct": 0.6,
    "operationsFundPct": 0.2,
    "developerSharePct": 0.2
  },
  "RESIDENT_SETTLEMENT": {
    "totalPct": 1.0,
    "communityFundPct": 0.6,
    "operationsFundPct": 0.2,
    "developerSharePct": 0.2
  },
  "VENDOR_SETTLEMENT": { ... },
  "VISITOR_SETTLEMENT": {
    "totalPct": 1.0,
    "communityFundPct": 0.8,
    "operationsFundPct": 0.2,
    "developerSharePct": 0.0
  },
  "DEPOSIT": { "totalPct": 0, ... },
  "TRANSFER": { ... },
  "PAYROLL": { "totalPct": 0, ... }
}
```

Percentages are stored as **human numbers**: `2.5` means 2.5%, not 0.025. The fee engine divides by 100 internally.

Transaction types not present in the rules object default to zero fee (`ZERO_RULE` in `fee-engine.ts`).

How `transferCredits()` works internally

`lib/ledger/service.ts:transferCredits()` does the following steps in order:

1. **Validates amount > 0** — throws if not
2. **Fetches the active fee schedule** (outside the DB transaction — read-only)
3. **Calculates fee split** via `calculateFee()` — pure function, no DB calls
4. **Opens `db.$transaction()`** : a. Looks up `fromWallet` and `toWallet` by ID b. Aggregates the sender's current balance (`SUM(amount) WHERE walletId = fromWalletId`) c. Throws `Insufficient balance` if sender balance < amount (automatic rollback) d. Fetches all 5 system wallets by `systemKey` e. Builds the entry array: `[fromWallet: -amount, toWallet: +netAmount, communityFund: +fee, operationsFund: +fee, developerShare: +fee]` f. ****Validates zero-sum****: `entries.reduce((acc, e) => acc.add(e.amount), 0)` must equal exactly 0 — throws if not (automatic rollback) g. Creates the `Transaction` row with all `LedgerEntry` rows via `createMany`
5. **After the transaction** (development only): calls `reconcileTreasury()` and throws if not balanced

System wallets and their roles

systemKey	Purpose
<code>treasury_reserve</code>	Backing reserve. Debited when K Credits are minted (deposits). Its negative balance represents total K Credits in circulation.
<code>community_fund</code>	Receives community portion of transaction fees. Funds community projects.
<code>operations_fund</code>	Receives operations portion of transaction fees. Funds day-to-day running costs.
<code>developer_share</code>	Receives developer portion of transaction fees.
<code>settlement_burn</code>	Receives K Credits when a settlement is executed. Credits flow here and are "burned" (removed from circulation), balancing the treasury.

The treasury balance visible in the admin dashboard is the `treasury_reserve` balance (which is negative, representing the reserve deployed).

Reconciliation logic

The reconciliation invariant:

```
sum(ALL LedgerEntry.amount) == sum(positive TREASURY_ADJUSTMENT entries)
```

`TREASURY_ADJUSTMENT` is the only one-sided entry type — it creates credits from fiat. All other transaction types are zero-sum pairs. Therefore, the total of all entries in the system should always equal the total K Credits ever issued.

`reconcileTreasury()` in `lib/ledger/reconciliation.ts` runs two Prisma aggregates:

- SUM of all `LedgerEntry.amount`
- SUM of positive `LedgerEntry.amount` where `transaction.type = 'TREASURY_ADJUSTMENT'`

If they differ, `isBalanced = false` and `discrepancy` shows the gap.

The live reconciliation page is at `/admin/treasury/debug`.

What can NEVER be done

These actions would corrupt the ledger. If you see code attempting any of these, treat it as a critical bug:

- UPDATE or DELETE on `LedgerEntry`** — ledger entries are immutable. Corrections are made with new entries.
- UPDATE or DELETE on `Transaction`** — same as above.
- Using JavaScript floating-point (`0.1 + 0.2`) for any financial calculation** — always use `Prisma.Decimal`.
- Calling `db.ledgerEntry.create()` or `db.transaction.create()` directly from a Server Action or API route** — all writes must go through `transferCredits()` or `recordDeposit()`.
- Fetching the fee schedule once and caching it in a module variable** — fees change; always fetch the active schedule at transaction time.
- Splitting `totalPct` as `gross × totalPct`** — use `sum(parts)` to compute `totalFee` to avoid rounding discrepancies that would violate the zero-sum invariant.

How to extend the fee schedule safely

To change fees or add a fee for a new transaction type:

- Do NOT update the existing `FeeSchedule` row. Create a new one.
- Set `effectiveAt` to a future timestamp (or `now()` for immediate effect).

3. Copy the full `rules` JSON from the current row and modify only what changes.
4. Insert via admin UI (Admin → Settings shows current schedule) or via a migration script.

Example SQL:

```
INSERT INTO "FeeSchedule" ("id", "effectiveAt", "rules", "createdBy", "createdAt")
VALUES (
  gen_random_uuid(),
  NOW(),
  '{"PURCHASE": {"totalPct": 3.0, "communityFundPct": 2.0, "operationsFundPct": 0.5, "developerSharePct": 0.5}, ...}::jsonb,
  "admin-user-id-here",
  NOW()
);
```

How to add a new transaction type

1. Add the new value to the `TransactionType` enum in `prisma/schema.prisma`.
2. Create and apply a migration: `pnpm exec prisma migrate dev --name add_new_txtype`.
3. Add a fee rule entry for the new type in a new `FeeSchedule` row (set to 0% if no fee).
4. Use the new type value in the `type` field when calling `transferCredits()`.
5. Update `lib/ledger/balance.ts` if the new type should contribute to `totalDeposited`, `totalEarned`, or `totalEligibleForConversion` in wallet summaries.
6. Add tests in `lib/ledger/__tests__/fee-engine.test.ts` for the new type's fee calculation.

8. Authentication and authorization

Clerk integration

Clerk handles authentication entirely. The Next.js app does not store passwords.

Sign-in flow: Users sign in via Clerk's hosted sign-in page at `/sign-in`. Clerk issues a session token stored as a cookie. All subsequent requests include this cookie.

Route protection: `proxy.ts` (Next.js 16's renamed `middleware.ts`) runs on every non-public request. It calls `clerkMiddleware` which calls `auth.protect()`. Unauthenticated requests are redirected to `/sign-in`.

Public routes (no auth required):

```
('/', '/sign-in(.*)', '/sign-up(.*)', '/api/webhooks(.*)',
'/access', '/login', '/dev-login', '/api/auth/token')
```

User table sync via webhook

When Clerk fires `user.created`, the webhook handler at `app/api/webhooks/clerk/route.ts`:

1. Verifies the `svix-id`, `svix-timestamp`, `svix-signature` headers using `CLERK_WEBHOOK_SECRET`
2. Checks if a user with the same email already exists (handles admin pre-creation case)
 - If yes: updates `clerkId` on the existing user
 - If no: creates a new `User` row with `memberId` (K-XXXXXXX), `role: VISITOR` by default
3. Creates an empty `Wallet` for the new user
4. On `user.updated` event: syncs `fullName`, `email`, `profilePhotoUrl`

The `requireRole()` helper

`lib/auth.ts:requireRole(role: Role | Role[])` is the authorization gate used at the top of every Server Action and sensitive page. It:

- 1. Calls `getCurrentUser()` which queries `db.user WHERE clerkId = clerkUserId`
- 2. If no user → `redirect('/sign-in')`
- 3. If user role not in allowed roles → `redirect('/')`
- 4. Returns the user object on success

Usage in Server Actions:

```
// Allow only Master Admin:
const admin = await requireRole('MASTER_ADMIN')

// Allow multiple roles:
const user = await requireRole(['RESIDENT', 'VISITOR'])

// Allow admin and master admin:
const admin = await requireRole(['MASTER_ADMIN', 'ADMIN'])
```

Where role checks live

- **Admin layout** (`app/(admin)/layout.tsx`): `requireRole('MASTER_ADMIN')` — only Master Admins see the admin area
- **All admin Server Actions** (`app/(admin)/_actions/*.ts`): explicit `requireRole` at the top of each action
- **Resident Server Actions** (`app/(resident)/_actions/*.ts`): `requireRole(['RESIDENT', 'VISITOR'])` or more specific
- **API routes**: `app/api/uploadthing/core.ts` uses `requireRole(['MASTER_ADMIN'])` for all upload endpoints

Clerk environments

Clerk has separate apps for development and production. You need two sets of keys:

- Development: uses `pk_test_...` and `sk_test_...` keys
- Production: uses `pk_live_...` and `sk_live_...` keys

The webhook URL must be updated in the Clerk dashboard each time the deployment URL changes.

9. Feature catalog

Phase 1 complete features

Feature	Route(s)	Key files	Status
Landing page with role redirect	<code>/</code>	<code>app/page.tsx</code>	Complete
Sign-in / sign-up	<code>/sign-in</code> , <code>/sign-up</code>	<code>app/(auth)/</code>	Complete
Founders page (public)	<code>/about/founders</code>	<code>app/about/founders/page.tsx</code>	Complete
Clerk webhook → User + Wallet creation	—	<code>app/api/webhooks/clerk/route.ts</code>	Complete

Admin sidebar navigation	<code>/admin/*</code>	<code>components/shared/admin-sidebar.tsx</code>	Complete
Admin dashboard with treasury stats	<code>/admin/dashboard</code>	<code>app/(admin)/admin/dashboard/</code>	Complete
Settlement approval queue	<code>/admin/approvals</code>	<code>app/(admin)/admin/approvals/</code>	Complete
Settlement execution with proof upload	<code>/admin/treasury</code>	<code>app/(admin)/admin/treasury/</code>	Complete
Deposit recording (fiat → K Credits)	<code>/admin/treasury</code>	<code>app/(admin)/admin/treasury/</code> , <code>lib/ledger/deposits.ts</code>	Complete
Treasury adjustment	<code>/admin/treasury</code>	<code>app/(admin)/admin/treasury/</code> , <code>app/(admin)/_actions/treasury.ts</code>	Complete
Live reconciliation debug page	<code>/admin/treasury/debug</code>	<code>app/(admin)/admin/treasury/debug/</code>	Complete
Account creation (invite via Clerk)	<code>/admin/accounts</code>	<code>app/(admin)/admin/accounts/</code> , <code>app/(admin)/_actions/accounts.ts</code>	Complete
Account suspend / restore	<code>/admin/accounts</code>	<code>app/(admin)/_actions/accounts.ts</code>	Complete
Role upgrade	<code>/admin/accounts</code>	<code>app/(admin)/_actions/accounts.ts</code>	Complete
Property registry	<code>/admin/properties</code>	<code>app/(admin)/admin/properties/</code>	Complete
Property detail (5 tabs)	<code>/admin/properties/[propertyId]</code>	<code>app/(admin)/admin/properties/[propertyId]/</code>	Complete
Installment management	<code>/admin/properties/[propertyId]</code>	<code>app/(admin)/admin/properties/[propertyId]/</code>	Complete
Owner / tenant assignment	<code>/admin/properties/[propertyId]</code>	<code>app/(admin)/_actions/properties.ts</code>	Complete
Community updates publishing	<code>/admin/community</code>	<code>app/(admin)/admin/community/</code>	Complete
Community voting management	<code>/admin/community</code>	<code>app/(admin)/admin/community/</code>	Complete
Issue management + reply	<code>/admin/community</code>	<code>app/(admin)/admin/community/</code>	Complete
Fee schedule viewer	<code>/admin/settings</code>	<code>app/(admin)/admin/settings/</code>	Complete
Resident wallet (home)	<code>/wallet</code>	<code>app/(resident)/wallet/</code>	Complete
Wallet balance hero card	<code>/wallet</code>	<code>app/(resident)/wallet/_components/</code>	Complete
Transaction history (recent + paginated)	<code>/wallet</code> , <code>/wallet/transactions</code>	<code>app/(resident)/wallet/</code>	Complete
Transaction PDF receipt	<code>/wallet/receipt/[txId]</code>	<code>app/(resident)/wallet/receipt/</code>	Complete

Settlement request form	<code>/wallet</code>	<code>app/(resident)/wallet/_components/settlement-request-sheet.tsx</code>	Complete
Settlement request list + timeline	<code>/wallet/settlements</code>	<code>app/(resident)/wallet/settlements/</code>	Complete
Cancel settlement (PENDING only)	<code>/wallet/settlements</code>	<code>app/(resident)/_actions/wallet.ts</code>	Complete
Property tab (ownership)	<code>/property</code>	<code>app/(resident)/property/</code>	Complete
Property tab (tenancy)	<code>/property</code>	<code>app/(resident)/property/</code>	Complete
Installment milestone strip + detail	<code>/property</code>	<code>app/(resident)/property/_components/</code>	Complete
Installment PDF receipt	<code>/property/receipt/[installmentId]</code>	<code>app/(resident)/property/receipt/</code>	Complete
Photo carousel with lightbox	<code>/property</code>	<code>app/(resident)/property/_components/</code>	Complete
Community updates feed + acknowledge	<code>/community</code>	<code>app/(resident)/community/</code>	Complete
Community voting (cast + results)	<code>/community</code>	<code>app/(resident)/community/</code>	Complete
In-app notifications	<code>/community</code>	<code>app/(resident)/community/</code>	Complete
Raise issue FAB + dialog	<code>/community</code>	<code>app/(resident)/community/_components/raise-issue-fab.tsx</code>	Complete
My issues list	<code>/community/issues</code>	<code>app/(resident)/community/issues/</code>	Complete
Profile tab (photo, QR card, KYC)	<code>/profile</code>	<code>app/(resident)/profile/</code>	Complete
Member QR code card	<code>/profile</code>	<code>app/(resident)/profile/_components/member-qr-card.tsx</code>	Complete
Unread notification badge	(tab bar)	<code>components/shared/resident-tab-bar.tsx</code>	Complete
iOS PWA install prompt	(global)	<code>components/shared/ios-install-prompt.tsx</code>	Complete
UploadThing file upload (proof, photos)	—	<code>app/api/uploadthing/</code> , <code>lib/uploadthing.ts</code>	Complete
Fee engine with banker's rounding	—	<code>lib/ledger/fee-engine.ts</code>	Complete
Ledger unit tests (36 tests)	—	<code>lib/ledger/__tests__/</code>	Complete

Not in Phase 1

These features are **architecturally supported** (no schema changes needed) but the UI/logic is not implemented:

Feature	Reason deferred
Push notifications (FCM/APNs/WebPush)	Notification DB rows exist; real-time delivery not wired
Vendor portal (dedicated UI)	Vendors have wallets; no vendor-specific routes
Payroll bulk workflows	<code>PAYROLL</code> transaction type exists; no admin UI for bulk runs
Document e-signing (DocuSign/HelloSign)	Contract URLs stored; in-app signing not integrated
In-app messaging (resident ↔ admin)	No chat model or UI
Map view (satellite/plot view)	No mapping library installed
Stripe / online payment gateway	Deposits are admin-recorded manually
Mobile native app (Expo/React Native)	Web PWA is Phase 1 target
Two-factor authentication	Clerk supports it; not enforced
Audit log viewer UI	All mutations DB-logged; no admin UI to browse
Bulk CSV export	Data accessible via Supabase; no in-app export
Vendor merchandise catalog	Not designed
Community bartering marketplace	<code>BARTER</code> type exists; no UI
Gate scanner UI	Not designed
Calendly integration	Not designed
Emergency broadcast module	Issue system covers this; no dedicated emergency route
Automated approval workflows	All approvals are manual by MASTER_ADMIN/ADMIN
Community posts moderation queue	Admin sees all issues; no dedicated moderation queue
Supermarket inventory	Not in scope
Admin operational analytics dashboards	Dashboard shows treasury; no analytics beyond that

10. Brand system

Color tokens (defined in `app/globals.css`)

All colors are OKLCH. Use the CSS variable names as Tailwind utilities (e.g., `text-karis-green-900` , `bg-karis-gold-500`).
Never use hex values or default Tailwind color classes (indigo, blue, gray) in this codebase.

Token	OKLCH	Use
<code>karis-green-50</code>	<code>oklch(0.97 0.02 155)</code>	Very light green tint
<code>karis-green-100</code>	<code>oklch(0.92 0.04 155)</code>	Light green tint (landing hero gradient)
<code>karis-green-500</code>	<code>oklch(0.45 0.08 155)</code>	Mid green
<code>karis-green-700</code>	<code>oklch(0.32 0.07 155)</code>	Dark green

<code>karis-green-900</code>	<code>oklch(0.20 0.05 155)</code>	Brand green — primary CTA, headings
<code>karis-gold-300</code>	<code>oklch(0.85 0.10 80)</code>	Light gold accent
<code>karis-gold-500</code>	<code>oklch(0.72 0.13 75)</code>	Primary gold — secondary CTA, active tab underline
<code>karis-gold-700</code>	<code>oklch(0.55 0.12 70)</code>	Dark gold — "K" currency prefix
<code>karis-stone-50</code>	<code>oklch(0.98 0.005 70)</code>	App background
<code>karis-stone-100</code>	<code>oklch(0.95 0.01 70)</code>	Card borders, muted
<code>karis-stone-300</code>	<code>oklch(0.85 0.015 70)</code>	Borders, dividers
<code>karis-stone-500</code>	<code>oklch(0.65 0.02 70)</code>	Secondary text, metadata
<code>karis-stone-900</code>	<code>oklch(0.22 0.01 70)</code>	Admin sidebar background , body text
<code>status-yellow</code>	<code>oklch(0.82 0.15 90)</code>	Warning / low severity
<code>status-orange</code>	<code>oklch(0.70 0.18 50)</code>	Medium severity
<code>status-red</code>	<code>oklch(0.58 0.21 25)</code>	High severity / destructive
<code>status-green</code>	<code>oklch(0.65 0.16 145)</code>	Success

Typography

Variable	Font	Usage
<code>--font-display</code> / <code>font-display</code>	Cormorant Garamond	"City of Karis" wordmark, hero titles, quote-style content
<code>--font-heading</code> / <code>font-heading</code>	Fraunces	h1–h3 headings
<code>--font-body</code> / <code>font-sans</code>	Inter	Everything else

Loaded via `next/font/google` in `app/layout.tsx` .

Voice rules (from `SKILL.md`)

- Brand voice: **elegant, calm, confident, welcoming**
- Avoid: corporate jargon, religious undertones, tech-bro language, real-estate clichés
- Prefer specific, grounded benefits: "off-grid solar with 72-hour battery backup" not "luxury living"
- Currency: always "K Credits" — not "credits", "tokens", or "coins"
- Roles: "Master Admin" (not "super admin"), "Resident" (capitalized)
- Transactions: "Settlement" for K → fiat; "Deposit" for fiat → K
- Numbers: `font-variant-numeric: tabular-nums` on all ledger/wallet displays
- Currency format: `K 1,250.00` — "K" in gold (`karis-gold-700`), always 2 decimal places

Where tokens are consumed

- `app/globals.css` — defined (source of truth)
- `app/layout.tsx` — fonts loaded
- All component files in `components/` and `app/` — consumed as Tailwind classes

Full brand reference

See `.claude/skills/cok-brand/SKILL.md` (relative to the Claude Main workspace root, not the website directory). This file is the authoritative brand specification used during development.

Forbidden UI patterns

- Emoji in production UI (except ₺ in country-specific contexts)
- Gradients except: gold gradient line on wallet hero (karis-gold only), landing page radial green tint
- Drop shadows beyond shadow-sm / shadow-md
- Icons from any library other than lucide-react
- Mixing USD \$ with K Credit K values in the same display

11. Operations runbook

Daily checklist

Task	How to do it	Expected result
Reconciliation check	Navigate to /admin/treasury/debug	All rows green, "Reconciliation OK" banner
Settlement queue	Navigate to /admin/approvals	Process any PENDING_APPROVAL requests
Clerk webhook health	https://dashboard.clerk.com → Webhooks → Recent Deliveries	Zero failed deliveries in last 24h
Stale KYC accounts	/admin/accounts → filter by PENDING_KYC	Follow up with any account stuck > 48h

Weekly checklist

Task	How to do it	Expected result
Settlement totals review	/admin/treasury → view executed settlements	Totals match expected treasury inflows
UploadThing storage check	https://uploadthing.com/dashboard	Under 2GB (free tier) — upgrade if approaching limit
Supabase backup verification	https://supabase.com/dashboard → Database → Backups	Automated backup ran successfully
Community maintenance	/admin/community	Close expired votes; resolve stale issues

Monthly checklist

Task	How to do it	Expected result
Fee schedule review	/admin/settings → Fee Schedule section	Rates match current leadership approval
Expired invitations	https://dashboard.clerk.com → Invitations	Revoke unclaimed invitations older than 30 days
Suspended accounts audit	/admin/accounts → filter by SUSPENDED	Decide: reinstate or permanently deactivate

System wallets check	<code>/admin/dashboard</code> → Treasury section	All 5 system wallets present, no unexpected balances
Dependency audit	<code>cd website && pnpm outdated</code>	Review any major version updates; test before applying
Security review	Check Clerk dashboard for failed sign-in attempts	No unusual patterns

Manual reconciliation commands

If the debug page shows a mismatch, run these SQL queries in the Supabase SQL editor:

```
-- Show total K Credits issued (positive TREASURY_ADJUSTMENT entries):
SELECT SUM(amount) AS total_issued
FROM "LedgerEntry" le
JOIN "Transaction" t ON t.id = le."transactionId"
WHERE t.type = 'TREASURY_ADJUSTMENT' AND le.amount > 0;

-- Show sum of ALL entries (should equal total_issued):
SELECT SUM(amount) AS sum_all FROM "LedgerEntry";

-- Find the discrepancy:
SELECT
  (SELECT SUM(amount) FROM "LedgerEntry" le
   JOIN "Transaction" t ON t.id = le."transactionId"
   WHERE t.type = 'TREASURY_ADJUSTMENT' AND le.amount > 0) AS total_issued,
  (SELECT SUM(amount) FROM "LedgerEntry") AS sum_all_entries;
```

Never fix a mismatch by deleting or updating rows. Create a compensating `TREASURY_ADJUSTMENT` entry after investigation. Contact the development team first.

12. Deployment and environments

Environments

Environment	Trigger	Database	Clerk
Development	<code>pnpm dev</code> on localhost	Supabase dev project (or local Docker PG)	Clerk dev app (pk_test_ keys)
Preview	Vercel auto-deploy on every non-main branch push	Same Supabase dev project (careful — shared DB)	Clerk dev app
Production	Vercel auto-deploy on push to <code>main</code>	Supabase prod project	Clerk prod app (pk_live_ keys)

Warning: Vercel preview deployments share the dev database unless you configure separate `DATABASE_URL` per environment in Vercel's environment variable settings (scoped to Preview).

Vercel configuration (`vercel.json`)

```
{
  "regions": ["iad1"],
  "buildCommand": "pnpm exec prisma generate && pnpm exec prisma migrate deploy && pnpm next build",
  "installCommand": "pnpm install --frozen-lockfile"
}
```

The build command runs three steps:

1. `prisma generate` — rebuilds the Prisma client in the CI environment
2. `prisma migrate deploy` — applies any pending migrations to the target database (safe, non-destructive — never resets)
3. `next build` — compiles the Next.js application

Deploying a change

```
# 1. Make changes locally
# 2. Verify everything passes
pnpm typecheck && pnpm lint && pnpm test && pnpm build

# 3. Commit and push to GitHub
git add -A
git commit -m "feat: your change"
git push origin main

# 4. Vercel auto-deploys within ~2 minutes
# Monitor: https://vercel.com/dashboard → your project → Deployments
```

Rolling back a deployment

```
# Option 1: Vercel dashboard
# Go to Deployments → find the last working deploy → "..." → Promote to Production

# Option 2: Git revert
git revert HEAD
git push origin main
```

First-time deployment checklist

1. Push the repo to GitHub (private):

```
git init # if not already a git repo
git add -A
git commit -m "feat: initial Phase 1 build"
gh repo create cok-app --private --source=. --remote=origin --push
```

2. Import at vercel.com → New Project → select `cok-app` → root directory: `website`
3. Add all 7 environment variables in Vercel → Project Settings → Environment Variables. Set them for Production, Preview, and Development as appropriate. Use production Clerk keys for Production.
4. First deploy runs automatically.
5. Update the Clerk webhook URL to the Vercel production URL:
 - Clerk dashboard → Webhooks → find the webhook → Edit endpoint URL → `https://YOUR_VERCEL_URL/api/webhooks/clerk`

- Events: `user.created` , `user.updated`
- Copy the new signing secret → update `CLERK_WEBHOOK_SECRET` in Vercel → redeploy

6. Run the production seed (once, from your local machine with `.env` pointing to the production DB):

```
pnpm seed:production
```

7. (Optional) Add custom domain:

- Vercel → Project → Domains → Add `app.cityofkaris.com`
- DNS: CNAME `app` → `cname.vercel-dns.com`
- SSL auto-provisions within ~1 minute

8. Smoke test:

- Sign in as Master Admin → `/admin/dashboard` should load with treasury cards
- Create a test account (Accounts → Create) — confirm Member ID generated
- Record a deposit — confirm wallet balance updates
- Sign in as the new resident — confirm wallet shows deposit

Secret rotation

When rotating credentials, update both the service and Vercel, then redeploy:

```
Clerk secret key:      Clerk → API Keys → Roll → update CLERK_SECRET_KEY in Vercel
Clerk webhook secret: Clerk → Webhooks → Roll → update CLERK_WEBHOOK_SECRET in Vercel
Resend API key:       Resend → API Keys → New → update RESEND_API_KEY in Vercel
UploadThing token:    UploadThing → API Keys → update UPLOADTHING_TOKEN in Vercel
Database password:    Supabase → Settings → Database → Reset → update DATABASE_URL in Vercel
```

After any Vercel env var change: Vercel → Deployments → Redeploy (last deployment).

13. Common tasks — how to

1. Add a new user role

Warning: Adding a role requires a database migration and code changes.

1. Add the new value to the `Role` enum in `prisma/schema.prisma` :

```
enum Role {
  MASTER_ADMIN
  ADMIN
  VENDOR
  RESIDENT
  VISITOR
  NEW_ROLE // ← add here
}
```

2. Run a migration:

```
pnpm exec prisma migrate dev --name add_new_role
```

3. Update `requireRole()` calls in Server Actions and layouts to include or exclude the new role.
4. Update the landing page redirect in `app/page.tsx` to handle the new role.
5. Update `proxy.ts` if the new role needs different route access.
6. Add the new role to the `createAccountAction` UI in `app/(admin)/admin/accounts/_components/create-account-dialog.tsx`.

2. Modify the fee schedule

Do NOT edit existing rows. Insert a new `FeeSchedule` row:

```
-- Run in Supabase SQL Editor
-- Copy the existing rules first, then modify:
SELECT rules FROM "FeeSchedule" ORDER BY "effectiveAt" DESC LIMIT 1;

-- Then insert a new row with updated rules:
INSERT INTO "FeeSchedule" ("id", "effectiveAt", "rules", "createdBy", "createdAt")
VALUES (
  gen_random_uuid(),
  NOW(), -- or a future timestamp: '2026-06-01 00:00:00'
  '{"PURCHASE": {"totalPct": 3.0, "communityFundPct": 2.0, "operationsFundPct": 0.5, "developerSharePct": 0.5}, ...}::jsonb,
  "your-user-id",
  NOW()
);
```

The new schedule takes effect immediately (or at the scheduled future time) with no code deployment needed.

3. Add a new transaction type

See [Section 7 — How to add a new transaction type](#).

4. Add a new property to the registry

Via the admin UI:

1. Go to `/admin/properties` → click "Add property"
2. Fill in: code (must be unique, e.g., `COMMERCIAL-C01`), type (`OWNERSHIP`/`RENTAL`/`ADMIN`), category, address
3. Add key-value specifications using the spec editor
4. After creation, go to the property detail page to add installments and assign owners/tenants

Via code (for seed data):

```
await db.property.create({
  data: {
    code: 'COMMERCIAL-C01',
    type: 'OWNERSHIP',
    category: 'COMMERCIAL',
    address: '12 Commerce Lane, Karis',
    totalPrice: new Prisma.Decimal(500000),
    specifications: { sqft: 2400, floors: 2 },
    photos: ['https://images.unsplash.com/...'],
  }
})
```

5. Publish a community update via admin UI

1. Go to `/admin/community` → click "New update"

2. Fill in: category, headline, message, optional photo URL
3. Click "Publish" → `publishUpdateAction()` runs:
 - Creates `CommunityUpdate` row
 - Creates `Notification` rows for all RESIDENT accounts
4. Residents see the update in their Community tab feed immediately (on next page load)

6. Investigate a user's wallet discrepancy

```
-- Step 1: Find the user's wallet ID
SELECT u.id, u.email, u."memberId", w.id AS wallet_id
FROM "User" u
JOIN "Wallet" w ON w."userId" = u.id
WHERE u.email = 'the-user@example.com';

-- Step 2: Get their current balance
SELECT SUM(amount) AS balance FROM "LedgerEntry" WHERE "walletId" = 'wallet-id-here';

-- Step 3: Get their full transaction history
SELECT
  t.id, t.type, t.description, t."createdAt",
  le.amount, le.description AS entry_desc
FROM "LedgerEntry" le
JOIN "Transaction" t ON t.id = le."transactionId"
WHERE le."walletId" = 'wallet-id-here'
ORDER BY t."createdAt" DESC;

-- Step 4: Verify running balance (should match UI)
SELECT
  t."createdAt", t.type, le.amount,
  SUM(le.amount) OVER (ORDER BY t."createdAt") AS running_balance
FROM "LedgerEntry" le
JOIN "Transaction" t ON t.id = le."transactionId"
WHERE le."walletId" = 'wallet-id-here'
ORDER BY t."createdAt";
```

7. Reset a user's password / re-issue magic link

1. Go to <https://dashboard.clerk.com> → Users → find the user
2. Click "Reset password" to send a password reset email
3. For magic-link users: go to Users → the user → "Send invitation" or "Reset" depending on sign-in method
4. Alternatively: admin can revoke the user's session, forcing re-authentication

8. Run a manual database backup

Supabase runs automated daily backups. For a manual snapshot:

1. Supabase dashboard → your project → Database → Backups → "Create backup"
2. For a local dump:

```
pg_dump "postgresql://USER:PASSWORD@HOST:5432/postgres" \
--no-acl --no-owner \
-f backup_$(date +%Y%m%d_%H%M%S).sql
```

9. Update brand colors or copy

1. Colors: edit the `@theme` inline block in `app/globals.css`. Use OKLCH values only. The Tailwind utility classes (`text-karis-green-900`) update automatically.
2. Fonts: update the `next/font/google` imports in `app/layout.tsx` and the CSS variable assignments in `globals.css`.
3. Copy: find the text in the component file and edit directly. Grep for it:

```
grep -r "Your wallet is being set up" app/ components/
```

4. Brand skill: the authoritative brand guide is `.claude/skills/cok-brand/SKILL.md` (in the Claude Main workspace root). Update it and the `globals.css` in sync.

10. Deploy a hotfix

```
# 1. Create a hotfix branch
git checkout -b hotfix/your-fix-name

# 2. Make the minimal change
# 3. Verify
pnpm typecheck && pnpm lint && pnpm test && pnpm build

# 4. Commit
git add -A
git commit -m "fix: description of the fix"

# 5. Push to trigger Vercel preview deploy
git push origin hotfix/your-fix-name

# 6. Test on the preview URL from Vercel

# 7. Merge to main
git checkout main
git merge hotfix/your-fix-name
git push origin main

# Vercel auto-deploys production
```

11. Onboard a new founding member end-to-end

1. **Create account** in Admin → Accounts → "Create account":
 - Full name, email address, role (typically RESIDENT)
 - System generates a Member ID (K-XXXXXX) and sends a Clerk invitation to the email
2. **Member accepts invitation:** they click the link in the email → Clerk creates their account → webhook fires → `clerkId` is set on the pre-created User row
3. **Record initial deposit** in Admin → Treasury → "Record deposit":
 - Select the new member
 - Enter fiat amount (equals K Credit amount — 1:1)
 - Select payment method, optionally upload proof
 - K Credits appear in the member's wallet
4. **Assign property** (if applicable) in Admin → Properties → find the property → Assign Owner:
 - Select the member
 - Set ownership percentage (usually 100)
 - Set contract date and optionally upload contract
5. **Add installments** in the property detail → Installments tab:
 - Add each payment milestone with number, due date, amount
6. **Verify:** Sign in as the new member or browse their account in Admin → Accounts → their account drawer to confirm: wallet balance, property assignment, member ID

12. Audit a settlement request from end to end

```
-- Step 1: Find the settlement request
SELECT
  sr.id, sr."userId", u.email, u."fullName",
  sr.amount, sr.status, sr.purpose,
  sr."createdAt", sr."approvedAt", sr."settledAt",
  sr."approvedBy", sr."settledBy", sr."proofUrl",
  sr."transactionId"
FROM "SettlementRequest" sr
JOIN "User" u ON u.id = sr."userId"
ORDER BY sr."createdAt" DESC;

-- Step 2: If settled, examine the transaction
SELECT
  t.id, t.type, t.description, t."feeScheduleId", t."createdAt",
  le."walletId", le.amount, w."systemKey", u.email
FROM "Transaction" t
JOIN "LedgerEntry" le ON le."transactionId" = t.id
LEFT JOIN "Wallet" w ON w.id = le."walletId"
LEFT JOIN "User" u ON u.id = w."userId"
WHERE t.id = 'transaction-id-here'
ORDER BY le.amount DESC;

-- Step 3: Verify zero-sum
SELECT t.id, SUM(le.amount) AS entry_sum
FROM "Transaction" t
JOIN "LedgerEntry" le ON le."transactionId" = t.id
WHERE t.id = 'transaction-id-here'
GROUP BY t.id;
-- Should return: entry_sum = 0.00000000
```

Check the Clerk dashboard to confirm the admin user ID matches the `approvedBy` / `settledBy` fields.

14. Known issues and technical debt

Bugs

#	Issue	Location	Severity	Fix
1	<code>declineSettlementAction</code> passes empty string for <code>declinedBy</code>	<code>app/(admin)/_actions/settlements.ts:40</code>	Low	Change <code>declinedBy: ''</code> to <code>declinedBy: admin.id</code> (requires <code>const admin = await requireRole(...)</code> line before the call)

Missing components

#	Issue	Impact	Fix
2	<code>components/ui/form.tsx</code> not installed	Forms use RHF directly (works fine); not a visible defect	Run <code>pnpm dlx shadcn@latest add form --overwrite</code> from <code>website/</code>

Data integrity / operational

#	Issue	Impact	Fix
---	-------	--------	-----

3	<code>pnpm seed:demo</code> has no idempotency guard	Running it twice creates duplicate demo transactions	Guard by checking for a sentinel transaction before seeding, or avoid running more than once per DB
4	<code>prisma generate</code> invalidated by <code>pnpm add</code>	TypeScript errors after installing new packages	Always run <code>pnpm exec prisma generate</code> after any <code>pnpm add</code>
5	Windows EPERM on <code>.next</code> rebuild	<code>pnpm build</code> may fail if <code>.next/</code> is locked	Run <code>rmdir /s .next</code> before production build if this occurs

Testing gaps

#	Issue	Impact	Fix
6	No integration tests (E2E)	<code>playwright-core</code> installed but not wired; server actions tested manually only	Wire up Playwright against a test database; write E2E tests for key flows
7	No browser QA was run during build	UI correctness verified by code reading only; no screenshot confirmation	Do a full manual walkthrough on a real device after deployment
8	No test for Clerk webhook handler	Webhook logic untested; covers User + Wallet creation	Add a Vitest test mocking the svix verification + Prisma client

Architecture / Phase 2 stubs

#	Issue	Notes
9	Resend API key required but email not actually sent	<code>lib/notifications/service.ts</code> creates DB rows; no email delivery is wired yet
10	<code>ADMIN</code> role has fewer permissions than <code>MASTER_ADMIN</code> but shares the admin sidebar	ADMIN cannot execute settlements or make treasury adjustments; the sidebar shows these routes but they will redirect
11	System wallets' negative balances not shown in UI	Treasury reserve goes negative as K Credits are minted; this is expected but might confuse operators
12	No rate limiting on Server Actions	High-frequency form submissions could cause DB load
13	Supabase free tier has limited concurrent connections	Session pooler mitigates this but there is a connection limit
14	<code>DIRECT_URL</code> env var required but unused (both point to session pooler)	<code>lib/env.ts</code> validates it, <code>prisma.config.ts</code> only uses <code>DATABASE_URL</code>

15. Path to Phase 2

Deferred features (estimated effort)

Feature	Effort	Notes
Push notifications (FCM/WebPush)	M (1–2 weeks)	Notification DB rows already exist; add delivery layer

Stripe online payment gateway	M (1–2 weeks)	Replace manual deposit recording
Vendor portal (dedicated UI)	M (2–3 weeks)	Build vendor-specific routes using existing wallet infrastructure
Payroll bulk workflows	S (3–5 days)	<code>PAYROLL</code> type exists; UI only
Document e-signing	M (1–2 weeks)	DocuSign or HelloSign API integration
Audit log viewer UI	S (3–5 days)	Query <code>Transaction</code> and <code>LedgerEntry</code> ; admin-facing read-only view
Two-factor authentication	S (1–2 days)	Clerk MFA configuration; enforcement policy
Bulk CSV export	S (2–3 days)	API route streaming CSV from Prisma queries
Mobile native app (Expo)	L (4–8 weeks)	Full Expo + React Native build targeting iOS/Android
KYC integration	M (2–3 weeks)	Integrate with identity verification provider
Real fiat rails (banking API)	L (4–8 weeks)	Automate settlement payouts; requires compliance work
In-app messaging	M (2–3 weeks)	Chat model + real-time layer (Pusher, Ably, or Supabase Realtime)
Map view	S–M (1–2 weeks)	Mapbox GL JS with property plot overlay
Gate scanner UI	M (1–2 weeks)	QR code scanner using Member ID
Bartering marketplace	M (2–3 weeks)	List/accept barter offers; use <code>BARTER</code> transaction type

Tier 1 → Tier 2 production readiness checklist

Before moving from Phase 1 (soft launch) to Phase 2 (full production):

- ☐ **KYC integration** — verify identity for all members before full financial permissions
- ☐ **Security audit** — external penetration test; review all Server Actions for authorization gaps
- ☐ **MFA enforcement** — require Clerk MFA for all MASTER_ADMIN and ADMIN accounts
- ☐ **Rate limiting** — protect Server Actions and API routes (Upstash Redis or Vercel Edge Rate Limit)
- ☐ **Error monitoring** — integrate Sentry or similar; alert on 5xx errors and ledger failures
- ☐ **Real backup test** — restore the database from a Supabase backup to a fresh project; verify data integrity
- ☐ **Load test** — stress-test the ledger service and settlement flow with concurrent requests
- ☐ **Compliance review** — consult with legal on financial regulations in Guyana for the K Credit economy
- ☐ **Terms of service** — add T&C and privacy policy pages; capture user consent at sign-up
- ☐ **Audit log UI** — operators must be able to see who did what and when without database access

Recommended sequencing

- Security audit + MFA enforcement (gate for any real financial activity)
- Error monitoring (Sentry)
- Stripe integration (replace manual deposits)
- KYC integration (required for full member onboarding)
- Push notifications (quality-of-life, high resident impact)
- Vendor portal (opens merchant ecosystem)
- Audit log UI (compliance requirement)
- Mobile native app (major UX investment)

16. Contacts and access

Fill this section in before handover.

Service	Account email / owner	Notes
GitHub repo	—	Repository owner
Vercel	—	Project admin
Supabase	—	Project owner
Clerk	—	Application owner
UploadThing	—	Account owner
Resend	—	Account owner
Domain registrar (cityofkaris.com)	—	DNS administrator
Google Fonts	(no account needed)	—

Role	Name	Email	Responsibility
Project owner	—	—	Business decisions, access management
Technical lead	—	—	Code changes, deployments, incidents
Operations admin	—	—	Daily checklists, settlement processing

Ongoing service costs (approx.):

Service	Free tier	Paid trigger
Vercel	Hobby is free; Pro ~\$20/mo	When you need team access or more build minutes
Supabase	Free up to 500MB DB, 2GB bandwidth	Upgrade at Pro ~\$25/mo when hitting limits
Clerk	Free up to 10,000 MAUs	~\$25/mo at growth
UploadThing	Free 2GB storage	Storage-based pricing above that
Resend	Free 3,000 emails/mo	~\$20/mo at higher volume

Appendix A: Full Prisma schema

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
```

```

}

// --- Enums ---

enum Role {
    MASTER_ADMIN
    ADMIN
    VENDOR
    RESIDENT
    VISITOR
}

enum AccountStatus {
    ACTIVE
    SUSPENDED
    PENDING_KYC
}

enum TransactionType {
    DEPOSIT
    PURCHASE
    TRANSFER
    BARTER
    PAYROLL
    RESIDENT_SETTLEMENT
    VENDOR_SETTLEMENT
    VISITOR_SETTLEMENT
    TREASURY_ADJUSTMENT
    FEE_SPLIT
    REVERSAL
}

enum SettlementStatus {
    PENDING_APPROVAL
    APPROVED
    DECLINED
    SETTLED
    CANCELLED
}

enum PropertyType {
    OWNERSHIP
    RENTAL
    ADMIN
}

enum PropertyCategory {
    COMMERCIAL
    RESIDENTIAL
    MIXED
}

enum IssueLevel {
    YELLOW
    ORANGE
    RED
}

enum IssueStatus {
    OPEN
    IN_PROGRESS
    RESOLVED
    CLOSED
}

// --- Core Identity ---

model User {
    id          String      @id @default(cuid())
    clerkId     String?     @unique
    memberId    String      @unique
    email       String      @unique
    fullName    String
    role        Role
    status      AccountStatus @default(PENDING_KYC)
    profilePhotoUrl String?
    introduction String?    @db.Text

```

```

kyc                Json?
createdAt          DateTime @default(now())
updatedAt          DateTime @updatedAt

wallet             Wallet?
ownedProperties    PropertyOwnership[]
rentedProperties   PropertyTenancy[]
notifications     Notification[]
issuesReported    Issue[] @relation("IssueReporter")
issuesAssigned    Issue[] @relation("IssueAssignee")
votes             VoteSubmission[]

@@index([email])
@@index([role])
@@index([status])
}

// — Wallet & Ledger —————

model Wallet {
  id          String      @id @default(cuid())
  userId      String?     @unique
  user        User?       @relation(fields: [userId], references: [id])
  isSystem    Boolean     @default(false)
  systemKey   String?     @unique
  entries     LedgerEntry[]
  createdAt   DateTime    @default(now())

  @@index([systemKey])
}

model FeeSchedule {
  id          String      @id @default(cuid())
  effectiveAt DateTime
  rules       Json
  createdBy   String
  createdAt   DateTime    @default(now())
  transactions Transaction[]

  @@index([effectiveAt])
}

model Transaction {
  id          String      @id @default(cuid())
  type        TransactionType
  description  String
  reference    String?
  feeScheduleId String?
  feeSchedule FeeSchedule? @relation(fields: [feeScheduleId], references: [id])
  initiatedBy  String?
  metadata     Json?
  createdAt   DateTime    @default(now())
  entries     LedgerEntry[]

  @@index([createdAt])
  @@index([type])
}

model LedgerEntry {
  id          String      @id @default(cuid())
  transactionId String
  transaction Transaction @relation(fields: [transactionId], references: [id])
  walletId    String
  wallet      Wallet      @relation(fields: [walletId], references: [id])
  amount      Decimal     @db.Decimal(20, 8)
  description  String?
  createdAt   DateTime    @default(now())

  @@index([walletId])
  @@index([transactionId])
}

// — Financial Operations —————

model Deposit {
  id          String      @id @default(cuid())
  userId      String

```

```

    fiatAmount    Decimal    @db.Decimal(20, 2)
    currency      String     @default("USD")
    paymentMethod String
    proofUrl      String?
    recordedBy    String
    transactionId String     @unique
    createdAt     DateTime   @default(now())

    @@index([userId])
}

model SettlementRequest {
  id          String      @id @default(cuid())
  userId      String
  amount      Decimal     @db.Decimal(20, 8)
  status      SettlementStatus @default(PENDING_APPROVAL)
  purpose     String?
  approvedBy  String?
  approvedAt  DateTime?
  declinedReason String?
  settledBy   String?
  settledAt   DateTime?
  proofUrl    String?
  transactionId String?
  createdAt   DateTime     @default(now())

  @@index([userId])
  @@index([status])
}

model TreasuryAdjustment {
  id          String      @id @default(cuid())
  amount      Decimal     @db.Decimal(20, 2)
  currency    String     @default("USD")
  reason      String
  recordedBy  String
  createdAt   DateTime   @default(now())
}

// — Property —————

model Property {
  id          String      @id @default(cuid())
  code        String      @unique
  type        PropertyType
  category    PropertyCategory
  address     String?
  totalPrice  Decimal?    @db.Decimal(20, 2)
  specifications Json?
  photos      String[]
  documents   Json?
  createdAt   DateTime     @default(now())

  ownerships  PropertyOwnership[]
  tenancies   PropertyTenancy[]
  installments PropertyInstallment[]
}

model PropertyOwnership {
  id          String      @id @default(cuid())
  propertyId  String
  userId      String
  ownershipPct Decimal     @default(100)
  contractDate DateTime
  contractUrl String?

  property Property      @relation(fields: [propertyId], references: [id])
  user      User          @relation(fields: [userId], references: [id])
  payments  PropertyPayment[]
}

model PropertyInstallment {
  id          String      @id @default(cuid())
  propertyId  String
  number      Int
  dueDate     DateTime
  amount      Decimal     @db.Decimal(20, 2)

```

```

    progressNote String?

    property Property @relation(fields: [propertyId], references: [id])
    payments PropertyPayment[]
}

model PropertyPayment {
  id String @id @default(cuid())
  installmentId String
  ownershipId String
  amount Decimal @db.Decimal(20, 2)
  paidAt DateTime
  proofUrl String?

  installment PropertyInstallment @relation(fields: [installmentId], references: [id])
  ownership PropertyOwnership @relation(fields: [ownershipId], references: [id])
}

model PropertyTenancy {
  id String @id @default(cuid())
  propertyId String
  userId String
  cycle String
  cyclePayment Decimal @db.Decimal(20, 2)
  contractDate DateTime
  contractUrl String?

  property Property @relation(fields: [propertyId], references: [id])
  user User @relation(fields: [userId], references: [id])
  cyclePayments TenancyCyclePayment[]
}

model TenancyCyclePayment {
  id String @id @default(cuid())
  tenancyId String
  cycleNumber Int
  amount Decimal @db.Decimal(20, 2)
  paidAt DateTime
  proofUrl String?

  tenancy PropertyTenancy @relation(fields: [tenancyId], references: [id])
}

// --- Community ---

model CommunityUpdate {
  id String @id @default(cuid())
  category String
  headline String
  message String @db.Text
  photoUrl String?
  publishedBy String
  publishedAt DateTime @default(now())

  acknowledgements UpdateAcknowledgement[]
}

model UpdateAcknowledgement {
  id String @id @default(cuid())
  updateId String
  userId String
  acknowledgedAt DateTime @default(now())

  update CommunityUpdate @relation(fields: [updateId], references: [id])

  @@unique([updateId, userId])
}

model Vote {
  id String @id @default(cuid())
  headline String
  description String @db.Text
  isOpen Boolean @default(true)
  createdBy String
  createdAt DateTime @default(now())
  closedAt DateTime?

```

```

    options      VoteOption[]
    submissions  VoteSubmission[]
}

model VoteOption {
  id            String @id @default(cuid())
  voteId        String
  label         String
  description   String @db.Text

  vote          Vote          @relation(fields: [voteId], references: [id])
  submissions   VoteSubmission[]
}

model VoteSubmission {
  id            String @id @default(cuid())
  voteId        String
  optionId      String
  userId        String
  submittedAt   DateTime @default(now())

  vote          Vote          @relation(fields: [voteId], references: [id])
  option         VoteOption @relation(fields: [optionId], references: [id])
  user           User         @relation(fields: [userId], references: [id])

  @@unique([voteId, userId])
}

model Issue {
  id            String @id @default(cuid())
  reporterId    String
  assigneeId    String?
  seriousness   IssueLevel
  urgency       IssueLevel
  category      String
  message       String @db.Text
  status        IssueStatus @default(OPEN)
  createdAt     DateTime @default(now())
  updatedAt     DateTime @updatedAt

  reporter User @relation("IssueReporter", fields: [reporterId], references: [id])
  assignee User? @relation("IssueAssignee", fields: [assigneeId], references: [id])
  replies IssueReply[]
}

model IssueReply {
  id            String @id @default(cuid())
  issueId       String
  authorId      String
  message       String @db.Text
  createdAt     DateTime @default(now())

  issue Issue @relation(fields: [issueId], references: [id])
}

model Notification {
  id            String @id @default(cuid())
  userId        String
  type          String
  title         String
  body          String?
  link          String?
  priority      String @default("yellow")
  readAt        DateTime?
  createdAt     DateTime @default(now())

  user User @relation(fields: [userId], references: [id])

  @@index([userId, readAt])
}

```

Appendix B: Environment variables reference

Variable	Purpose	Where to obtain	Dev example	Prod source
<code>DATABASE_URL</code>	Primary DB connection (session pooler)	Supabase → Project Settings → Database → Connection pooler → Session mode → URI	<code>postgresql://postgres.PROJECT:PASSWORD@aws-0-us-east-1.pooler.supabase.com:5432/postgres</code>	Supabase prod prod → same
<code>DIRECT_URL</code>	Direct DB connection (used by Prisma for migrations in current setup, set to same as <code>DATABASE_URL</code>)	Same as above (same URL used in the current setup)	Same as <code>DATABASE_URL</code>	Same as <code>DATABASE_URL</code>
<code>NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY</code>	Clerk client-side key (exposed to browser)	Clerk dashboard → API Keys	<code>pk_test_...</code>	<code>pk_live_...</code>
<code>CLERK_SECRET_KEY</code>	Clerk server-side key (never exposed to browser)	Clerk dashboard → API Keys	<code>sk_test_...</code>	<code>sk_live_...</code>
<code>CLERK_WEBHOOK_SECRET</code>	Validates Clerk webhook signatures	Clerk dashboard → Webhooks → find endpoint → Signing secret	<code>whsec_...</code>	Same pair prod Clerk app
<code>RESEND_API_KEY</code>	Transactional email (required at server boot, email not yet sent in Phase 1)	Resend dashboard → API Keys	<code>re_...</code>	Resend prod key
<code>UPLOADTHING_TOKEN</code>	File upload service authentication	UploadThing dashboard → API Keys	<code>eyJ...</code>	UploadThing prod key
<code>NEXT_PUBLIC_CLERK_SIGN_IN_URL</code>	Redirect URL for sign-in (optional, default <code>/sign-in</code>)	Hardcode in <code>.env.example</code>	<code>/sign-in</code>	<code>/sign-in</code>

<code>NEXT_PUBLIC_CLERK_SIGN_UP_URL</code>	Redirect URL for sign-up (optional, default <code>/sign-up</code>)	Hardcode in <code>.env.example</code>	<code>/sign-up</code>	<code>/sign-up</code>
--	---	---------------------------------------	-----------------------	-----------------------

Note: `DATABASE_URL` must point to the **session pooler (port 5432)**. Do not use the transaction pooler (port 6543) — Prisma uses prepared statements that are incompatible with transaction mode.

Note: The app validates `DATABASE_URL`, `DIRECT_URL`, `CLERK_SECRET_KEY`, `CLERK_WEBHOOK_SECRET`, `RESEND_API_KEY`, `UPLOADTHING_TOKEN`, and `NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY` at server startup via `lib/env.ts`. The server will throw and refuse to start if any of these are missing.

Appendix C: API surface

API routes

Path	Method	Purpose	Auth required	Notes
<code>/api/webhooks/clerk</code>	POST	Clerk user lifecycle events	svix signature	Creates User + Wallet on <code>user.created</code> ; syncs on <code>user.updated</code>
<code>/api/uploadthing</code>	GET, POST	UploadThing file upload handler	MASTER_ADMIN (per route config)	Handles proof-of-payment (4MB), property photos (8MB×10), contracts (16MB×5)
<code>/wallet/receipt/[transactionId]</code>	GET	Stream transaction PDF receipt	Resident (session)	Returns <code>application/pdf</code> via <code>@react-pdf</code>
<code>/property/receipt/[installmentId]</code>	GET	Stream installment PDF receipt	Resident (ownership verified)	Returns <code>application/pdf</code>

Admin Server Actions

All in `app/(admin)/_actions/`. All require `requireRole(['MASTER_ADMIN', 'ADMIN'])` unless noted.

File	Function	Purpose	Role required
<code>settlements.ts</code>	<code>approveSettlementAction(settlementId)</code>	Approve a PENDING_APPROVAL settlement	MASTER_ADMIN or ADMIN
<code>settlements.ts</code>	<code>declineSettlementAction(settlementId, reason)</code>	Decline with reason	MASTER_ADMIN or ADMIN
<code>settlements.ts</code>	<code>executeSettlementAction(settlementId, proofUrl?)</code>	Execute APPROVED settlement (debits ledger)	MASTER_ADMIN only

deposits.ts	recordDepositAction(userId, fiatAmount, paymentMethod, currency, proofUrl?)	Mint K Credits for a user	MASTER_ADMIN or ADMIN
treasury.ts	recordTreasuryAdjustmentAction(amount, currency, reason)	Manual treasury adjustment	MASTER_ADMIN only
accounts.ts	createAccountAction(data)	Create user + send Clerk invitation	MASTER_ADMIN or ADMIN
accounts.ts	suspendAccountAction(userId)	Set account status to SUSPENDED	MASTER_ADMIN or ADMIN
accounts.ts	restoreAccountAction(userId)	Set account status to ACTIVE	MASTER_ADMIN or ADMIN
accounts.ts	upgradeRoleAction(userId, newRole)	Change user role	MASTER_ADMIN or ADMIN
properties.ts	createPropertyAction(data)	Create a property record	MASTER_ADMIN or ADMIN
properties.ts	addInstallmentAction(propertyId, data)	Add installment to a property	MASTER_ADMIN or ADMIN
properties.ts	assignOwnerAction(propertyId, userId, data)	Create PropertyOwnership	MASTER_ADMIN or ADMIN
properties.ts	assignTenantAction(propertyId, userId, data)	Create PropertyTenancy	MASTER_ADMIN or ADMIN
properties.ts	addPropertyPaymentAction(installmentId, ownershipId, data)	Record installment payment	MASTER_ADMIN or ADMIN
community.ts	publishUpdateAction(data)	Create CommunityUpdate + notifications	MASTER_ADMIN or ADMIN
community.ts	createVoteAction(data)	Create Vote + options + notifications	MASTER_ADMIN or ADMIN
community.ts	closeVoteAction(voteId)	Close an open vote	MASTER_ADMIN or ADMIN
community.ts	replyToIssueAction(issueId, message)	Add IssueReply + notify reporter	MASTER_ADMIN or ADMIN
community.ts	updateIssueStatusAction(issueId, status)	Update issue lifecycle status	MASTER_ADMIN or ADMIN
community.ts	assignIssueAction(issueId)	Assign issue to current admin	MASTER_ADMIN or ADMIN

Resident Server Actions

All in `app/(resident)/_actions/` . All require `requireRole(['RESIDENT', 'VISITOR'])` unless noted.

File	Function	Purpose	Role required
------	----------	---------	---------------

wallet.ts	requestSettlementAction(amount, purpose)	Create PENDING_APPROVAL settlement request	RESIDENT or VISITOR
wallet.ts	cancelSettlementRequestAction(requestId)	Cancel a PENDING_APPROVAL request	RESIDENT or VISITOR (own request)
wallet.ts	loadMoreTransactionsAction(walletId, cursor)	Cursor-paginated transaction list	RESIDENT or VISITOR
community.ts	acknowledgeUpdateAction(updateId)	Record UpdateAcknowledgement	RESIDENT or VISITOR
community.ts	castVoteAction(voteId, optionId)	Create VoteSubmission	RESIDENT or VISITOR
community.ts	raiseIssueAction(data)	Create Issue + notify admins	RESIDENT or VISITOR
community.ts	markAllNotificationsReadAction()	Set readAt on all unread notifications	RESIDENT or VISITOR
profile.ts	updateIntroductionAction(text)	Update user introduction field	RESIDENT or VISITOR
profile.ts	updateProfilePhotoAction(url)	Update profilePhotoUrl	RESIDENT or VISITOR

Appendix D: Useful queries

Run these in the Supabase SQL Editor (Dashboard → SQL Editor).

1. All wallets with current balances

```
SELECT
  w.id AS wallet_id,
  w."systemKey",
  w."isSystem",
  u.email,
  u."fullName",
  u."memberId",
  SUM(le.amount) AS balance
FROM "Wallet" w
LEFT JOIN "User" u ON u.id = w."userId"
LEFT JOIN "LedgerEntry" le ON le."walletId" = w.id
GROUP BY w.id, w."systemKey", w."isSystem", u.email, u."fullName", u."memberId"
ORDER BY w."isSystem" DESC, balance DESC;
```

2. Settlement requests by status

```
SELECT
  sr.id, u.email, u."fullName",
  sr.amount, sr.status, sr.purpose,
  sr."createdAt", sr."approvedAt", sr."settledAt",
  sr."declinedReason"
FROM "SettlementRequest" sr
JOIN "User" u ON u.id = sr."userId"
WHERE sr.status = 'PENDING_APPROVAL' -- change to filter by status
ORDER BY sr."createdAt" ASC;
```

3. Full transaction history for a user (by email)

```

SELECT
  t.id AS tx_id, t.type, t.description,
  le.amount,
  t."createdAt"
FROM "Transaction" t
JOIN "LedgerEntry" le ON le."transactionId" = t.id
JOIN "Wallet" w ON w.id = le."walletId"
JOIN "User" u ON u.id = w."userId"
WHERE u.email = 'devon@example.com'
ORDER BY t."createdAt" DESC;

```

4. Manual treasury reconciliation

```

SELECT
  (
    SELECT COALESCE(SUM(le.amount), 0)
    FROM "LedgerEntry" le
    JOIN "Transaction" t ON t.id = le."transactionId"
    WHERE t.type = 'TREASURY_ADJUSTMENT' AND le.amount > 0
  ) AS total_issued,
  (
    SELECT COALESCE(SUM(amount), 0)
    FROM "LedgerEntry"
  ) AS sum_all_entries,
  (
    SELECT COALESCE(SUM(le.amount), 0)
    FROM "LedgerEntry" le
    JOIN "Transaction" t ON t.id = le."transactionId"
    WHERE t.type = 'TREASURY_ADJUSTMENT' AND le.amount > 0
  ) - (
    SELECT COALESCE(SUM(amount), 0)
    FROM "LedgerEntry"
  ) AS discrepancy;
-- discrepancy should be 0.00000000

```

5. Top 10 spenders (by total debits)

```

SELECT
  u.email, u."fullName", u."memberId",
  SUM(ABS(le.amount)) AS total_spent
FROM "LedgerEntry" le
JOIN "Wallet" w ON w.id = le."walletId"
JOIN "User" u ON u.id = w."userId"
WHERE le.amount < 0
      AND w."isSystem" = false
GROUP BY u.id, u.email, u."fullName", u."memberId"
ORDER BY total_spent DESC
LIMIT 10;

```

6. All pending KYC accounts

```

SELECT id, email, "fullName", "memberId", "createdAt"
FROM "User"
WHERE status = 'PENDING_KYC'
ORDER BY "createdAt" ASC;

```

7. Active fee schedule (current rules)

```

SELECT id, "effectiveAt", rules, "createdBy", "createdAt"
FROM "FeeSchedule"
WHERE "effectiveAt" <= NOW()
ORDER BY "effectiveAt" DESC
LIMIT 1;

```

8. Transactions in the last 7 days with fee breakdown

```

SELECT
  t.id, t.type, t.description, t."createdAt",
  le."walletId",
  le.amount,
  w."systemKey",
  u.email
FROM "Transaction" t
JOIN "LedgerEntry" le ON le."transactionId" = t.id
LEFT JOIN "Wallet" w ON w.id = le."walletId"
LEFT JOIN "User" u ON u.id = w."userId"
WHERE t."createdAt" >= NOW() - INTERVAL '7 days'
ORDER BY t."createdAt" DESC, t.id;

```

9. Zero-sum check — verify specific transaction

```

SELECT t.id, SUM(le.amount) AS entry_sum
FROM "Transaction" t
JOIN "LedgerEntry" le ON le."transactionId" = t.id
WHERE t.id = 'PASTE_TRANSACTION_ID_HERE'
GROUP BY t.id;
-- entry_sum must be 0.00000000 for all non-TREASURY_ADJUSTMENT transactions

```

10. Community update acknowledgement rates

```

SELECT
  cu.id, cu.headline, cu."publishedAt",
  COUNT(ua.id) AS acknowledged_count,
  (SELECT COUNT(*) FROM "User" WHERE role IN ('RESIDENT', 'VISITOR') AND status = 'ACTIVE') AS total_residents,
  ROUND(
    COUNT(ua.id)::numeric /
    NULLIF((SELECT COUNT(*) FROM "User" WHERE role IN ('RESIDENT', 'VISITOR') AND status = 'ACTIVE'), 0) * 100,
    1
  ) AS ack_rate_pct
FROM "CommunityUpdate" cu
LEFT JOIN "UpdateAcknowledgement" ua ON ua."updateId" = cu.id
GROUP BY cu.id, cu.headline, cu."publishedAt"
ORDER BY cu."publishedAt" DESC;

```

11. Open issues by seriousness

```

SELECT
  i.id, i.seriousness, i.urgency, i.category, i.status,
  u.email AS reporter_email,
  a.email AS assignee_email,
  i."createdAt",
  COUNT(r.id) AS reply_count
FROM "Issue" i

```

```

JOIN "User" u ON u.id = i."reporterId"
LEFT JOIN "User" a ON a.id = i."assigneeId"
LEFT JOIN "IssueReply" r ON r."issueId" = i.id
WHERE i.status NOT IN ('RESOLVED', 'CLOSED')
GROUP BY i.id, u.email, a.email
ORDER BY
  CASE i.seriousness WHEN 'RED' THEN 1 WHEN 'ORANGE' THEN 2 ELSE 3 END,
  i."createdAt" ASC;

```

12. Treasury flow by transaction type (last 30 days)

```

SELECT
  t.type,
  COUNT(DISTINCT t.id) AS transaction_count,
  SUM(CASE WHEN le.amount > 0 AND w."systemKey" = 'treasury_reserve' THEN -le.amount ELSE 0 END) AS credits_issued,
  SUM(CASE WHEN le.amount < 0 AND w."systemKey" = 'treasury_reserve' THEN -le.amount ELSE 0 END) AS credits_returned
FROM "Transaction" t
JOIN "LedgerEntry" le ON le."transactionId" = t.id
JOIN "Wallet" w ON w.id = le."walletId"
WHERE t."createdAt" >= NOW() - INTERVAL '30 days'
GROUP BY t.type
ORDER BY credits_issued DESC;

```